HUET Séverin
GIORDANI Enzo
PETROVIC Hugo

26/03/2026

Projet SAÉ - Refonte de l'Application de Placement

RENDU Final de Projet

Table des matières

1. Introduction 2
2. Technologies utilisées 2
3. Informations sur le projet 2
4. Base de données 3
4.1 Bilan des évolutions de la base de données 3
4.2 Stratégie de migration 3
4.3 Évolutions structurelles détaillées 3
5. Liste détaillée des fonctionnalités du projet 4
6. Répartition détaillée des tâches 6
6.1 Séverin - Chef de projet & Data Lead (~40%) 6
6.2 Enzo - Lead Developer Architecture MVC (~30%) 6
6.3 Hugo - Developpeur Full-Stack & Integration PDF (~30%) 7
6.4 Conclusion sur la GPO 7
7. Conclusion du projet 7
8. Annexe : Manuel d'utilisation du projet 8
Étape 1 - Connexion à l'application 8
Étape 2 - Page d'accueil (Dashboard) 8
Étape 3 - Gestion des données (module administrateur) 9
3.1 Gestion des salles (?action=gest_salle) 10
3.2 Création d'une salle (?action=crea_salle) 11
3.3 Gestion des enseignants, matières, promotions et groupes 11
Étape 4 - Outil de placement (?action=util_placement) 12
Étape 5 - Export PDF 13
Étape 6 — Déconnexion 13

1. Introduction

Ce projet de SAE 4.01 a pour objet la refonte complète de l'application web InfoPlacement, utilisée par le département informatique de l'IUT de Metz pour automatiser le placement des étudiants lors des évaluations. L'application existante reposait sur une architecture procédurale, une base de données non normalisée présentant des anomalies de redondance, et un mécanisme d'authentification obsolète et vulnérable.

Les objectifs de la refonte ont été les suivants :

- **Architecturaux** : Migrer vers un patron de conception MVC (Modèle-Vue-Contrôleur) afin de séparer clairement la logique métier, l'accès aux données et la présentation.
- **Base de données** : Normaliser le schéma relationnel jusqu'à la Quatrième Forme Normale (4FN) afin d'éliminer les dépendances trop redondantes et les anomalies de mise à jour.
- **Sécurité** : Remplacer le hachage MD5 par BCrypt, systématiser l'utilisation de requêtes préparées PDO, et introduire une gestion robuste des sessions.
- **Maintenabilité** : Réorganiser l'ensemble des fichiers source en dossiers thématiques (controllers/, models/, views/, utils/, ajax/) pour faciliter l'évolution et la collaboration en équipe.

2. Technologies utilisées

Langage serveur : PHP 8.2.20
SGBD : MariaDB 10.3.39
Accès aux données : PDO (PHP Data Objects) avec requêtes préparées et paramètres nommés
Sécurité des mots de passe : BCrypt via password_hash() / password_verify() (PHP natif)
Génération de PDF : Bibliothèque ezpdf (class.ezpdf.php / class.pdf.php) — FPDF 1.86 en réserve
Front-end : HTML5, CSS
Interactions dynamiques : JavaScript / AJAX (appels aux endpoints dédiés dans ajax/)
Gestion de projet : Git / GitHub, Trello (méthode agile avec sprints hebdomadaires)
Environnement de déploiement : Serveur DevWeb de l'IUT
Encodage : UTF-8mb4 (support des caractères spéciaux et emojis)

3. Informations sur le projet

Lien devweb de la version finale :
https://devweb.iutmetz.univ-lorraine.fr/~e40250u/sae401/Placement

Lien devbdd de la base de données :
https://devweb.iutmetz.univ-lorraine.fr/phpmyadmin/index.php?route=/database/structure&db=e40250u_sae401

Comptes utilisateurs :

Rôle	Login	Mot de passe en clair
Administrateur	roka	admin123
Enseignant standard	bougnath	prof123

Lien GitHub : lien (TAG Vfinal)
Lien Trello (lien d'invitation) : lien

4. Base de données

Un Modèle Conceptuel de Données (MCD) et un Modèle Logique de Données (MLD) sont disponibles en annexe pour illustrer cette architecture.

4.1 Bilan des évolutions de la base de données

La refonte de la base de données a constitué un axe majeur du projet. L'objectif était de corriger les violations des formes normales identifiées lors de l'analyse de l'existant, en particulier des dépendances multivaluées qui entraînaient des redondances et des anomalies de mise à jour.

4.2 Stratégie de migration

La migration a été réalisée en trois phases distinctes, via trois scripts SQL versionnés dans le dossier BDD/Script/ :

1. **01_creation_schema_v2.sql** (DDL — Structure & Optimisation)
Mise en place du nouveau schéma dans une zone de staging avec le préfixe v2_ pour éviter les conflits avec le schéma en production. Ce script introduit les types de données optimisés (INT UNSIGNED, TINYINT(1)), les contraintes d'intégrité référentielle (FOREIGN KEY avec politiques ON DELETE CASCADE ou RESTRICT), ainsi que des index de performance sur les colonnes stratégiques.

2. **02_migration_donnees.sql** (DML — Transfert & Assainissement)
Script de transfert intelligent des données. On utilise COALESCE pour récupérer les anciens id_mat, une sous-requête anti-orphelins, et la clause DISTINCT pour éliminer les doublons issus des produits cartésiens de l'ancienne structure.

3. **03_bascule_finale.sql** (Déploiement final)
Suppression de l'ancien schéma dans l'ordre des dépendances, puis RENAME TABLE simultané des tables v2_ vers leurs noms définitifs. L'opération de renommage étant atomique en MariaDB, ce script garantit une continuité de service totale.

4.3 Évolutions structurelles détaillées

Tables conservées et modifiées :

Table	Évolution
enseignant	Champ pass porté à varchar(255) (BCrypt) ; valeur par défaut de admin passée à 0 ; contrainte UNIQUE ajoutée sur login
devoir	Champ id_mat rendu optionnel (DEFAULT NULL) ; suppression de la liaison directe aux groupes/promotions (déplacée vers tables de jonction)
etudiant	Ajout de mob_reduite, premier_rang, tiers_temps (TINYINT(1)) pour la gestion de l'accessibilité et du tiers-temps
salle	Liaison ajoutée vers la table plan via id_plan
batiment	Typage renforcé ; champ adresse ad_bat ajouté

Tables de jonction créées (normalisation 4FN) :

Table	Rôle
devoir_groupe	Remplace l'ancien produit cartésien entre devoirs et groupes
devoir_promo	Liaison explicite devoir ↔ promotion
devoir_salle	Liaison explicite devoir ↔ salle de surveillance
surveille	Table de jonction enseignant ↔ devoir pour la surveillance
enseigne	Table de jonction enseignant ↔ matière

Vues SQL créées :

Vue	Utilité
v_groupe_effectif	Calcul optimisé de l'effectif par groupe, évite les sous-requêtes répétitives
v_salle_capacite	Calcul de la capacité disponible par salle selon son plan

Tables supprimées ou fusionnées :
Les anciennes tables de liaison dénormalisées (produits cartésiens implicites entre devoirs, groupes et salles) ont été remplacées par les tables de jonction explicites listées ci-dessus, conformément aux exigences de la 4FN.

5. Liste détaillée des fonctionnalités du projet

Ajoutées à l'existant :

- **Architecture MVC avec routeur centralisé :**
  La nouvelle version introduit un front controller (index.php) qui centralise les routes métier, les contrôles d'accès et les erreurs 404, alors que l'ancienne version chargeait les pages de façon procédurale.
- **Séparation claire des responsabilités :**
  La logique est découpée en controllers, models et views, ce qui facilite les tests, la maintenance et l'évolution fonctionnelle.
- **API AJAX structurée pour le placement :**
  Ajout de points d'entrée dédies (ajax_add_combi, ajax_suppr_combi, ajax_affiche_combi, ajax_intervertir, ajax_info_devoir) avec réponses JSON homogènes.
- **Workflow de placement en étapes explicites :**
  Formalisation du processus en placement_stage1 a placement_stage4 avec conservation d'état en session ($_SESSION['up']) et validations intermédiaires.
- **Renforcement de l'authentification :**
  Passage à password_hash/password_verify (Bcrypt) et regeneration d'ID de session après connexion (session_regenerate_id), ce qui augmente significativement la sécurité.
- **Mécanisme de changement de mot de passe force :**
  Ajout d'une route et d'une vue dédiées (changer_mdp) pour imposer le changement du mot de passe quand il est par défaut à la création d'un enseignant.
- **Migration de hash legacy :**
  Présence d'un script de migration des mots de passe MD5 vers Bcrypt (scripts/migrate_md5_to_bcrypt.php).

- **Export PDF fiabilise (FPDF) :**
  Remplacement pratique de flux PDF fragiles par une génération plus robuste avec FPDF (liste, émargement, plan), en ligne avec l'export enseignants déjà stable.

Supprimées de l'existant :

- **Chargement dynamique direct des pages par nom de fichier :**
  L'ancien mécanisme (content.php + include($p.".php")) a été retiré au profit d'un routage explicite.
- **Logique métier dans les vues procédurales :**
  Les traitements SQL/session embarqués dans les pages d'interface (up_stage*.php, pages de gestion) ont été déplacés dans les controleurs/modeles.
- **Authentification basée sur MD5 :**
  Suppression du schéma de vérification md5(...) utilisé dans l'ancien projet.
- **Dépendance forte a des flux PDF legacy (ezPDF) :**
  L'approche historique, plus sensible aux problèmes d'headers/encodage, est désormais contournée/remplacée pour les exports principaux.
- **Couplage implicite iframe/page unique pour le placement :**
  L'ancien mode procédural basé sur inclusions successives est remplacé par un parcours route par route plus explicite.

Modifiées, avec justification des évolutions :

- **Authentification (conservée mais modernisée)**
  Avant : vérification login/mot de passe en MD5.
  Après : vérification Bcrypt + meilleure hygiène de session.

Réduction du risque de compromission des mots de passe et alignement sur les pratiques de sécurité actuelles.

- **Gestion des comptes enseignants (conservée, mieux encadrée)**
  Avant : création/modification en procédural, contrôle de flux mélangé à l'affichage.
  Après : contrôleur dédie (EnseignantController) avec validations et messages unifies.

Meilleure maintenabilité et meilleure lisibilité fonctionnelle.

- **Moteur de placement (conserve, structure et renforce)**
  Avant : algorithme procédural lie aux pages et sessions disperses.
  Après : orchestration par PlacementController, vérifications de capacite, gestion de combinaisons, interversions via AJAX.

Robustesse métier accrue (cohérence des capacités, meilleure gestion des erreurs utilisateur), réduction des effets de bord.

- **Accès aux données (modifie pour sécurité)**
  Avant : requetés parfois interpolées directement dans les scripts.
  Après : usage prédominant de requêtes préparées dans les modelés.

Diminution du risque d'injection SQL et meilleure réutilisabilité des accès BDD.

- **Exports PDF (conserves fonctionnellement, refondus techniquement)**
  Avant : génération historique mélangée (ezPDF) et dépendante de variables de session legacy.
  Après : génération FPDF plus déterministe et mieux contrôlée (liste, émargement, plan).

Fiabilité d'ouverture des fichiers PDF, compatibilité navigateur, simplification de maintenance.

- **Navigation et ergonomie générale (conservée, rendue explicite)**
  Avant : navigation par pages procédurales et conventions implicites (p=).
  Après : routes nommées (action=), écrans d'étape plus clairs, messages d'erreur plus visibles.

Meilleure expérience utilisateur et meilleure compréhensibilité du flux.

6. Répartition détaillée des tâches

6.1 Séverin - Chef de projet & Data Lead (~40%)

- Responsabilités principales (architecture data / backend)

- Refonte complète de la base de données et normalisation.
- Migration et sécurisation de l'accès aux données via PDO.
- Contribution à la mise en place du cadre MVC sur les couches structurelles (routage + modélisation).
- Rédaction des comptes-rendus hebdomadaires de suivi.

- Implémentation

- Migration SQL/PDO Placement/config/connexion.php (connexion unique centralisée).
- Construction de la couche modèle, avec classes dédiées dans Placement/models/ :
- Passage d'un code ancien mélangeant logique et SQL vers une séparation métier plus propre.
- Appui sur le schéma final de référence BDD/e40250u_sae401_vfinal.sql pour garantir la cohérence entre modèles et BDD.

- Autres taches

- Rédaction continue des livrables de pilotage comptes-rendus et suivi d'avancement.

6.2 Enzo - Lead Developer Architecture MVC (~30%)

- Responsabilités principales (socle applicatif)

- Conception et implémentation du cœur MVC (tâche la plus longue et la plus structurante).
- Transformation du code procédural "spaghetti" vers une architecture modulaire.
- Mise en place de la séparation des responsabilités : controllers / models / views.

- Implémentation

- Front-controller et routage central dans Placement/index.php :
  Routage explicite action=...,
  Gestion des routes métier et AJAX,
  Contrôle des accès et redirections.
- Base commune des contrôleurs dans Placement/controllers/Controller.php (rendu de vues, garde admin, JSON).
- Construction des fichiers de controleurs metier dans Placement/controllers/ :
- Sécurisation de l'authentification au niveau architecture :
  Logique de session et flux auth dans AuthController.php,
  Passage a password_hash / password_verify dans models/EnseignantModel.php.

- Autres taches

- Assistance ponctuelle des autres membres sur les points bloquants d'intégration MVC.
- Validation de la cohérence inter-couches (routeur -> controleur -> modele -> vue).

6.3 Hugo - Developpeur Full-Stack & Integration PDF (~30%)

- Responsabilités principales

- Prise en charge du module Gestion des salles (création, modification, visualisation, parcours multi-étapes).
- Intégration et fiabilisation des exports PDF (liste, emargement, plan).
- Corrections frontend/structurelles, dont suppression des mécanismes obsolètes (frames / affichages legacy).

- Implémentation

- Module salles dans :
  Placement/controllers/SalleController.php,
  Placement/views/salle/
- Refonte de l'export PDF dans :
  Placement/public/export_pdf.php,
  Placement/public/export_pdf_ens.php,
  Utilisation de la librairie Placement/libs/fpdf186/.
- Rationalisation par rapport à l'ancien export procédural :
  Nouveau flux route action=export_pdf vers export dédie et stabilise.

- Evolution visuelle et structurelle sur les écrans de placement :
  CSS/JS de Placement/public/css/ et Placement/public/javascript/,
  Retrait progressif des logiques de rendu anciennes basées sur sous-pages/iframes du legacy.

- Autres taches

- Débogage de régressions d'affichage et de compatibilité navigateurs.
- Ajustements ergonomiques pour la lecture des grilles et des sorties PDF.

6.4 Conclusion sur la GPO

La charge de travail a été effectivement répartie de manière équitable selon une logique 40/30/30, non pas figée mais adaptée en continu a la complexité de rénovation du projet. Cette dynamique agile a renforcé la complémentarité des rôles :
Pilotage data et sécurisation, construction du socle MVC, et intégration full-stack metier.
En conséquence, l'équipe a maintenu la cohésion et réglé les risques techniques.

7. Conclusion du projet

Ce projet a permis de transformer une application web procédurale en une architecture MVC structurée, maintenable et sécurisée. Les bénéfices de cette refonte sont multiples.

Maintenabilité : La séparation stricte des responsabilités entre contrôleurs, modèles et vues permet désormais à chaque développeur d'intervenir sur une couche sans risquer d'impacter les autres. L'organisation des fichiers en dossiers thématiques (controllers/, models/, views/, ajax/, utils/) rend le projet immédiatement lisible pour tout nouveau contributeur.

Sécurité : Deux améliorations majeures ont été apportées. D'une part, le remplacement du hachage MD5 par l'algorithme BCrypt élimine la vulnérabilité aux attaques, BCrypt intègre un sel aléatoire unique par hash et est intentionnellement coûteux en calcul, ce qui rend les attaques par force brute impossible ou beaucoup trop longues.
D'autre part, la systématisation des requêtes préparées PDO avec paramètres nommés supprime toute possibilité d'injection SQL.

Base de données : La normalisation en 4FN élimine les redondances et anomalies de mise à jour de l'ancien schéma, garantissant la cohérence des données à long terme. La stratégie de migration en trois phases avec bascule atomique démontre une maîtrise des contraintes d'exploitation en environnement de production.

Ce projet constitue ainsi une démonstration concrète des bonnes pratiques du développement web. Nous avons mis en pratique toutes les bonnes manières que nous avons appris durant ses années à l'IUT. Alliant rigueur architecturale, conscience des enjeux de sécurité et gestion de projet agile.

8. Annexe : Manuel d'utilisation du projet

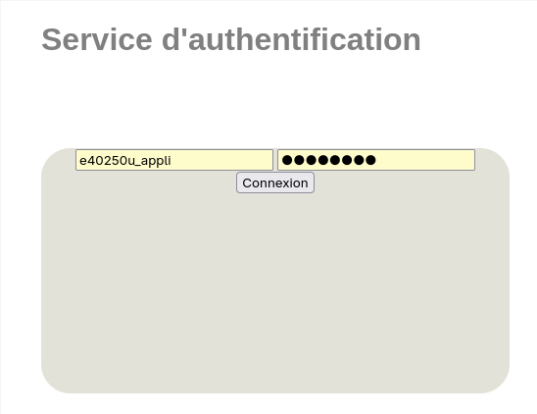
Ce manuel est destiné aux utilisateurs de l'application InfoPlacement (enseignants et administrateurs). Il décrit le parcours d'utilisation de l'interface.

Étape 1 - Connexion à l'application

Accédez à l'application via le lien devweb fourni en section 3. La page de connexion s'affiche automatiquement pour tout utilisateur non authentifié.
Saisissez votre login dans le premier champ.
Saisissez votre mot de passe dans le second champ.
Cliquez sur le bouton Connexion.

En cas d'identifiants incorrects, un message générique s'affiche : "Identifiants incorrects." — ce message ne précise pas volontairement si c'est le login ou le mot de passe qui est erroné, afin d'éviter l'énumération des comptes.

Note : *Si votre mot de passe est le mot de passe initial par défaut (declic), l'application vous redirigera automatiquement vers la page de changement de mot de passe avant toute autre action.*



Étape 2 - Page d'accueil (Dashboard)

Après connexion, vous êtes redirigé vers la page d'accueil. Celle-ci présente un résumé de l'application et donne accès à la barre de navigation principale.

La barre de navigation donne accès aux modules suivants :

Gestion des données (bâtiments, salles, départements, promotions, groupes, matières, enseignants)
Placement (outil de placement automatisé des étudiants en salle)
Déconnexion

Étape 3 - Gestion des données (module administrateur)

Les modules de gestion sont accessibles uniquement aux utilisateurs disposant du rôle Administrateur (admin = 1).

3.1 Gestion des salles (?action=gest_salle)

Affiche la liste de toutes les salles enregistrées. Pour chaque salle, des actions sont disponibles : visualiser le plan, modifier la configuration, ou supprimer la salle

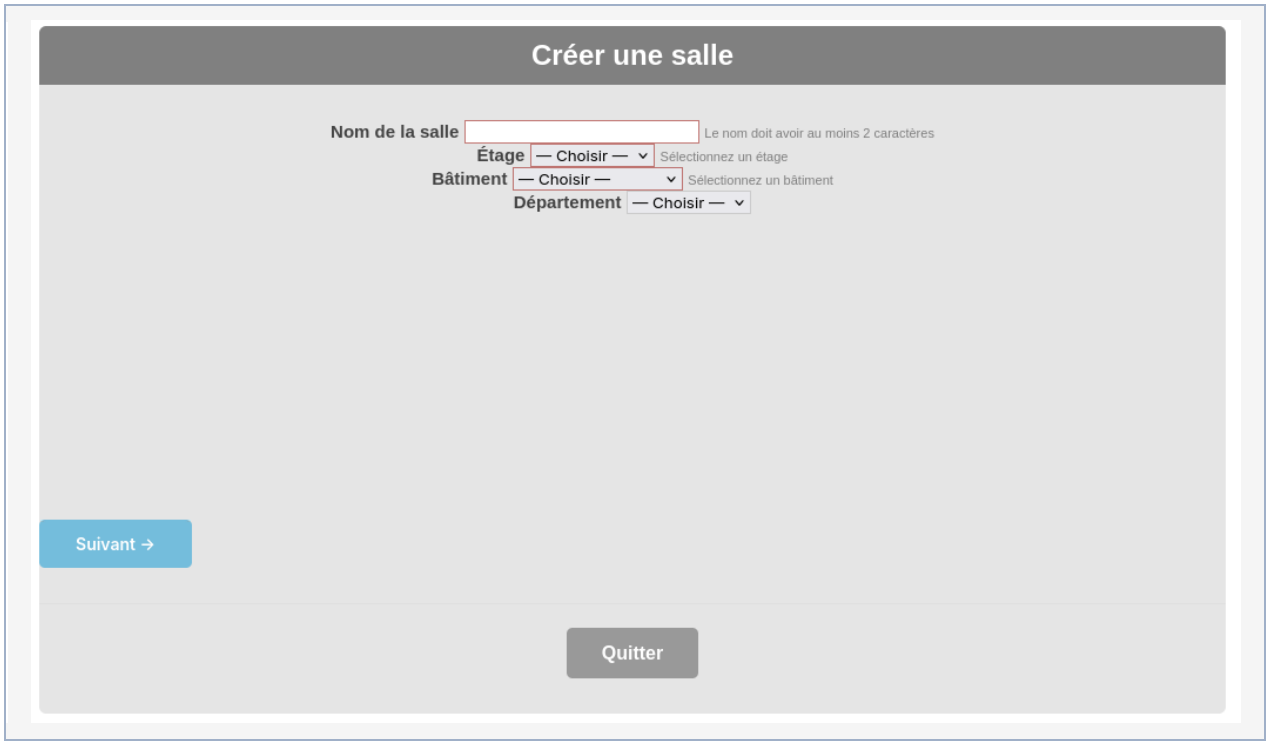
Gestion des salles

Créer une salle | Modifier une salle

NOM	BÂTIMENT	ÉTAGE	DÉPARTEMENT	CAPACITÉ	ACTIONS
Amphi 2	Lettres et Langues	0	INFO	211	
Amphi 3	Lettres et Langues	0	INFO	212	
Amphi A	IUT de Metz	0	INFO	150	
Amphi A (1sur2)	IUT de Metz	0	INFO	150	
Amphi B	IUT de Metz	0	INFO	150	
Amphi B (1sur2)	IUT de Metz	0	INFO	150	
B09	IUT de Metz	0	INFO	56	
E23	IUT de Metz	2	INFO	32	
E23 - tout	IUT de Metz	2	INFO	32	
E36	IUT de Metz	3	INFO	40	
E37	IUT de Metz	3	INFO	30	
F01	IUT de Metz	0	INFO	74	
F06	IUT de Metz	0	INFO	79	
F33	IUT de Metz	3	INFO	30	
F36	IUT de Metz	3	INFO	30	
F39	IUT de Metz	3	INFO	16	
test2	IUT de Metz	1	INFO	19	
tset	IUT de Metz	1	INFO	4	

3.2 Création d'une salle (?action=crea_salle)

La création d'une salle se fait en plusieurs étapes guidées (wizard multi-étapes) :
Étape 1 (cs_stage1.php) : Saisie du nom de la salle et sélection du bâtiment.
Étape 2 (cs_stage2.php) : Définition des dimensions de la grille (nombre de rangées et de colonnes).
Étape 3 (cs_stage3.php) : Configuration case par case du plan de la salle (places disponibles, places PMR, allées).
Étape 4 (cs_stage4.php) : Confirmation et enregistrement.



3.3 Gestion des enseignants, matières, promotions et groupes

Chacun de ces modules suit le même patron CRUD (Create, Read, Update, Delete) :
Un tableau liste les enregistrements existants.
Un formulaire permet d'ajouter un nouvel élément.
Des icônes d'action (modification, suppression) sont disponibles sur chaque ligne.

Étape 4 - Outil de placement (?action=util_placement)

C'est la fonctionnalité centrale de l'application. Elle permet de générer automatiquement le placement des étudiants dans les salles pour un devoir donné.

Le processus se déroule en plusieurs étapes guidées :
Sélection du devoir, de la promotion et/ou du groupe à placer. (up_stage1.php)

Sélection de la ou des salles disponibles pour le devoir. L'application affiche la capacité de chaque salle en fonction de son plan. (up_stage2.php)

Exécution de l'algorithme de placement. Il tient compte des besoins d'accessibilité des étudiants (mob_reduite, premier_rang, tiers_temps) pour attribuer les places en priorité. Le résultat est affiché sous forme de grille correspondant au plan de la salle. (up_stage3.php)

Validation du placement. Des interversions manuelles de places sont possibles via une interface AJAX (endpoint ajaxIntervertion.php). (up_stage4.php)

Cliquez sur deux places pour les intervertir.

Étape 5 - Export PDF

Une fois le placement validé, deux exports PDF sont disponibles via la bibliothèque ezpdf :
Export salle (export_pdf.php) : Génère un document PDF avec le plan de la salle et les noms des étudiants à chaque place. Destiné à être affiché en salle le jour de l'épreuve.
Export enseignant (export_pdf_ens.php) : Génère une liste nominative triée par enseignant surveillant. Destiné à la remise aux surveillants.

Plan - Amphi A

2026-03-27 - 10:00 - Duree: 10:00

GRAÏNE Y.		PIZETTE B.		COLASSE N.		BONKOSKI A.		BARRE A.		FADILY I.		KHOUCHI I.		SCHAAL O.
DENOYELLE V.		CORONA N.		GAÜZÈRE J.		BOSCAROL F.		ADAM M.				ROUX M.		HEYTIENNE M.
ISLER M.				STILLE E.		PLANCHER M.		GINOT A.		ADOLPH N.		JOLY A.		THIS E.
GIORDANI E.		DARCIAUX Y.		TRIBUT E.		FEIPEL A.		PRANDATO J.		KIRCHER N.		MARCHAL A.		BAYIHA-MBA D.
BAILLY C.		EL AAMERY I.		ROUMILI R.		CHAOUCHI A.		ARSLAN A.		PERON E.		PETROVIC H.		LUPO L.
TAVERNI A.		LADURELLE A.		FREY T.		GIRRES A.		TOK M.		DOBOS L.		LAURI M.		LEONARD--R E.
SALVADORI T.		BENMOUFFEK A.		SWIATOKA M.		MALLINGER R.		LAMBERT Q.				BARETH A.		ZOLLER M.
QUENTIN B.		OUKARA A.		BENDEKKICH S.		SCHOU L.		BOGUET T.		FUSCIELLO B.		SCHUMACHER T.		
				WADDELL S.				GOEPFERT-B M.		TOPAL F.		FEISTHAUER S.		TIRONI B.
HUET S.		ISHAK Y.		ALICI T.		BOUHOUCH C.				TUNC Y.		SIMON L.		RICHET F.

Étape 6 — Déconnexion

Cliquez sur le lien Déconnexion dans la barre de navigation. L'application détruit la session PHP côté serveur, supprime le cookie de session côté navigateur, et redirige vers la page de connexion.